

(1.) JavaScript – Einstieg & Grundlagen

📌 Was ist JavaScript?

- Dynamisch typisierte Skriptsprache, ursprünglich für Webseiten.
- Läuft im **Browser** (z. B. via `<script>`) oder im **Backend** über **Node.js**.
- Hat sich zu einer vollständigen Programmiersprache entwickelt (Client + Server).

📦 JavaScript im HTML

Inline JavaScript:

```
<script>
  function sayHello() {
    alert("Hallo Raphael!");
  }
</script>
```

Externe Datei:

```
<script src="main.js"></script>
```

1.1 📦 Variablen

Schlüsselwort	Beschreibung	Überschreibbar?
`var`	Veraltet, funktionaler Scope	✅
`let`	Modern, Block-Scope	✅
`const`	Konstante Referenz (nicht Inhalt)	❌

Wichtig:

`const` schützt nur die Referenz – nicht den Inhalt.

```
const person = { name: "Raphael" };
person.name = "Anna"; // ✅ erlaubt
person = {};          // ❌ nicht erlaubt
```

Typische Fehlerquellen:

- `var` in Schleifen: wird nicht blockweise begrenzt → Problem!
- `const` muss direkt initialisiert werden
- `const array = [];` → Inhalte dürfen geändert werden

1.2 🧠 Datentypen

Grundtypen in JavaScript:

- `string`, `number`, `boolean`, `undefined`, `null`
- `object`, `array`, `function`

Typprüfung mit `typeof`:

```
typeof "Hallo"      // 'string'
typeof 42           // 'number'
typeof true         // 'boolean'
```

```
typeof undefined      // 'undefined'
typeof null           // 'object' (! historischer Bug)
typeof [1,2,3]        // 'object'
typeof {a:1}          // 'object'
typeof function(){}   // 'function'
```

1.3 🧮 Funktionen

Klassische Funktion (Declaration)

```
function greet(name) {
  return "Hi " + name;
}
```

Ausdruck mit `const`:

```
const greet = function(name) {
  return "Hi " + name;
};
```

Pfeilfunktion (Arrow Function):

```
const greet = (name) => {
  return `Hi ${name}`;
};

// Kurzform
const greetQuick = name => `Hi ${name}`;
```

Hinweis: Arrow Functions haben **kein eigenes** `this`. (Hierzu noch erweitern)

Vergleich: `function` vs. `const` (Arrow Function)

Aspekt	`function`	`const` mit Arrow Function
Hoisting	✅ Ja	❌ Nein
Überschreibbar	✅ Ja	❌ Nein
Typ	Deklaration	Ausdruck

Scope	Funktional	Blockbasiert
`this`	Eigenes	Erbt
Typischer Einsatz	Methoden	Callbacks, Kurzfunktionen

1.4 🔁 Schleifen & Bedingungen

```
for (let i = 0; i < 3; i++) {
  console.log(i);
}

if (x > 5) {
  console.log("Größer als 5");
}

array.forEach(item => console.log(item));
```

Weitere Schleifenarten:

```
for (const fruit of fruits) {
  console.log(fruit);
}

for (const key in person) {
  console.log(key, person[key]);
}
```

Vergleichsoperatoren

Operator	Bedeutung
`==`	Vergleich mit Typumwandlung
`===`	Strenger Vergleich ✅

1.5 🧍 Objekte & Arrays

🔷 Struktur & Unterschiede

```
const person = { name: "Raphael", age: 24};        // Objekt
const fruits = ["apple", "banana", "cherry"];      // Array
```

Merkmal	Objekt ({}-Syntax)	Array ([]-Syntax)
Aufbau	Schlüssel–Wert-Paare	Geordnete Liste
Zugriff	`obj.key` / `obj["key"]`	`arr[0]` / `arr["1"]`
Reihenfolge	keine garantiert	feste Reihenfolge
Methoden	`Object.keys()` , `...`	`.map()` , `.push()` , `.filter()`
Nutzung	Eigenschaften, Datenstruktur	Listen, Datenmengen

🔍 Zugriff & Erweiterung

```
person.name          // "Raphael"
person["name"]       // auch gültig

fruits[0];           // "apple"
fruits["1"];         // "banana" (automatisch konvertiert)

person.hobby = "Tanzen";  // neue Eigenschaft

const newPerson = { ...person, age: 25};  // kopieren & ändern (flach)
```

🛠️ Objekt-Helfer (kurz)

```
Object.keys(person);     // ["name", "age"]
Object.values(person);   // ["Raphael", 24"]
Object.entries(person);  // [["name", "Raphael"], ["age", 24]]
```

⚙️ Array-Methoden

📥 Hinzufügen / Entfernen

```
fruits.push("kiwi");     // Ende hinzufügen
```

```
fruits.pop();          // letztes Element entfernen
```

🔍 Suchen & Prüfen

```
fruits.includes("apple"); // true
fruits.indexOf("banana"); // 1
fruits.length;          // 3
```

🔁 Verwandeln mit `.map()`

```
fruits.map((fruit, i) => ({
  id: i,
  name: fruit.toUpperCase()
}));
```

- `.map()` erstellt ein neues Array
- Du bekommst automatisch `element` & `index`

```
array.map(el => el.length);      // nur Element
array.map((_, i) => i * 2);      // nur Index
```

🧪 `Array.from()` – Umwandlung

```
const abschnitte = document.querySelectorAll("p");
const array = Array.from(abschnitte);
```

- Macht „array-ähnliche“ Objekte (NodeLists etc.) verarbeitbar
- Danach kannst du `.map()`, `.filter()` etc. verwenden

📌 Merken

- Objekte = ungeordnete Schlüssel-Wert-Paare
- Arrays = geordnete Listen mit Indizes
- `typeof [1, 2, 3]` → `"object"` (!)
- `"1"` wird bei Arrays zu `1` (Index), bei Objekten zu `"1"` (Key)
- `.map()` gibt immer ein neues Array zurück

- Spread (`...`) kopiert nur **flach**

1.6 🔍 Scope & Sichtbarkeit

Was ist "Scope"?

Der Scope bestimmt, **wo im Code eine Variable gültig ist**.

Die wichtigsten Scopes:

Scope-Typ	Beschreibung	Gilt für
Global Scope	Überall sichtbar	Alles außerhalb
Function Scope	Nur in Funktionen sichtbar	`var`, `function`
Block Scope	Nur im Block sichtbar `{}`	`let`, `const`

Beispiel:

```
let x = 10;

function test() {
  let y = 5;

  if (true) {
    let z = 1;
    console.log(x, y, z); // ✅
  }

  console.log(z); // ❌
}

console.log(y); // ❌
```

1.7 🧠 Closures, Scope Chains & funktionales Denken

Was ist ein Closure?

Ein **Closure** ist eine Funktion, die sich **Variablen aus einem äußeren Scope merkt**, auch wenn der äußere Kontext bereits beendet wurde.

Beispiel:

```
function begruessung(name) {
  return function() {
    console.log("Hallo, " + name);
  }
}

const hallo = begruessung("Raphael");
hallo(); // ➜ "Hallo, Raphael"
```

Scope Chain

```
let global = "A";

function außen() {
  let mitte = "B";

  function innen() {
    let lokal = "C";
    console.log(global, mitte, lokal); // A B C
  }

  innen();
}
```

Wofür sind Closures gut?

1. Zustand speichern:

```
function counter() {
  let zahl = 0;
  return function() {
    zahl++;
    console.log(zahl);
```

```
  }
}
```

2. Funktionsgeneratoren:

```
function multiplikator(x) {
  return function(y) {
    return x * y;
  }
}
```

3. Verzögerte Ausführung (`setTimeout`, Events):

```
function baueTimer(nachricht) {
  setTimeout(function() {
    console.log("Nachricht: " + nachricht);
  }, 1000);
}
```

1.8 🧱 Klassen, Konstruktoren & Module

Klassen in JavaScript

```
class Person {
  constructor(name, alter) {
    this.name = name;
    this.alter = alter;
  }

  vorstellen() {
    console.log(`Ich heiße ${this.name} und bin ${this.alter} Jahre alt.`);
  }
}
```

Alternativ: Objekt-Factory

```
function createPerson(name, alter) {
  return {
    name,
    alter,
    vorstellen() {
      console.log(`Ich heiße ${this.name} und bin ${this.alter} Jahre alt.`);
    }
  };
}
```

Wann sind Klassen sinnvoll?

Situation	Empfehlung
Viele gleichartige Objekte	✅ Klasse / Factory
Vererbung nötig	✅ Klasse
Einmalige Struktur	🔷 Funktion reicht
Kleine Scripts / Prototypen	🔷 Keine Klasse nötig

Module (`import` / `export`)

person.js:

```
export class Person {
  constructor(name) {
    this.name = name;
  }

  hallo() {
    console.log("Hallo " + this.name);
  }
}
```

main.js:

```
import { Person } from './person.js';
```

```
const ich = new Person("Raphael");
ich.hallo();
```

👉 Wichtig: im HTML:

```
<script type="module" src="main.js"></script>
```

Typische Fehler:

- `this` falsch verstanden (v. a. in Event-Handlern)
- Klassen genutzt, obwohl einfacher ginge
- Kein `type="module"` gesetzt
